

Technical Design Document

Service Mobility Dashboard

Full-Stack Final Project

1. Technical Overview

The Service Mobility Dashboard is a full-stack web application built for service dispatchers who manage field-service vehicles and daily service tasks.

The application uses a hybrid Next.js architecture in which operational data and authentication are handled on the server, while interactive map behavior and route visualization are handled in the browser.

The main technology stack is:

- Next.js 16 using the App Router.
- React 19.
- TypeScript.
- Tailwind CSS 4.
- Supabase/PostgreSQL for service-task data.
- Supabase Auth for authentication.
- Mapbox GL JS / react-map-gl for map visualization.
- Mapbox Optimization API for vehicle-route optimization.
- Google Places API for address autocomplete and place-coordinate lookup.
- react-hot-toast for non-blocking error notifications.
- Vitest for unit testing.
- Playwright for End-to-End testing.

The application is designed as a read-only decision-support layer over normalized operational data mirrored from an existing organizational source system. The upstream operational system - for example, an ERP platform such as SAP - remains the authoritative source of truth.

2. High-Level Architecture

The system is divided into three primary layers.

Client Layer

Runs in the user's browser and is responsible for:

- Rendering the interactive map.
- Displaying vehicle routes and service stops.
- Managing vehicle visibility and focus.
- Displaying task information.
- Searching addresses through Google Places.
- Managing recent address searches.
- Requesting Mapbox route optimization.
- Displaying recommendation results.
- Managing interactive UI state.

Next.js Server Layer

Runs on the application server and is responsible for:

- Authentication verification.
- Session handling.
- Loading daily service tasks.
- Generating assignment recommendations.
- Validating client-controlled input.
- Protecting access to operational data.

Data Layer

Supabase/PostgreSQL stores normalized, mirrored service-task records received from the organization's upstream operational system. The application uses the database primarily for authenticated READ operations.

```

Organizational source system / ERP
(e.g., SAP)
|
| Integration / synchronization layer
v
Supabase/PostgreSQL (service_tasks mirror)
^
| Server Actions
|
Next.js Server Layer
^
|
Web Browser ----> Google Places API
|
+-----> Mapbox APIs

```

Note: Source-system integration and synchronization are outside the scope of the current application. The dashboard assumes that normalized service-task records are available in `service_tasks`.

3. Project Folder Structure

```

fullstack_final_project/
|-- src/
|   |-- app/
|   |-- actions/
|   |-- components/
|   |-- hooks/
|   |-- lib/
|   |-- types/
|   |-- proxy.ts
|-- supabase/
|   |-- migrations/
|   |-- seed-e2e.sql
|-- tests/
|   |-- unit/
|   |-- e2e/
|-- playwright.config.ts
|-- vitest.config.mts
|-- package.json
|-- next.config.ts

```

- `src/app` - Next.js App Router pages and root layout.
- `src/actions` - Server Actions for authentication and secure database operations.
- `src/components` - interactive React UI components.
- `src/hooks` - reusable client-side stateful logic, primarily route optimization.
- `src/lib` - shared utilities and Supabase/Mapbox integrations.
- `src/types` - TypeScript representation of the database schema.
- `supabase/migrations` - database permission, Row Level Security, and indexing changes.
- `supabase/seed-e2e.sql` - controlled test fixture used to refresh the recommendation E2E data relative to the current date.
- `tests` - unit and End-to-End automated tests.

4. Application Routes

Dashboard Route (/)

The main service mobility dashboard. `src/app/page.tsx` is a Server Component.

Its responsibilities are:

1. Read the requested date from the URL.

2. Validate the date.
3. Fall back to the current service date when necessary.
4. Retrieve the day's service tasks through `getDailyTasks()`.
5. Render date navigation.
6. Pass the retrieved data to the client-side `MapDashboard`.

```
/?date=YYYY-MM-DD
```

The date in the URL makes the selected workday bookmarkable and compatible with browser Back/Forward navigation while ensuring the server retrieves only the requested day.

Login Route (/login)

Contains the authentication form. Users authenticate using Supabase Auth. There is intentionally no public registration page because accounts represent authorized organizational employees.

5. Main Components

MapDashboard

- Groups service tasks by vehicle.
- Prepares route-optimization requests.
- Renders the Mapbox map and service-task markers.
- Renders optimized route lines.
- Manages vehicle visibility and focus.
- Selects and displays individual tasks.
- Manages the searched draft location.
- Requests and displays vehicle/date recommendations.
- Navigates to recommended workdays.
- Handles route failures and dashboard errors.

DatePicker

- Previous-day and next-day navigation.
- Calendar selection and month navigation.
- Jump to today.
- Keyboard navigation and accessibility behavior.
- Updates the date query parameter through Next.js navigation.

AddressSearch

- Uses Google Places API (New) for address autocomplete.
- Begins autocomplete only after at least three characters.
- Debounces requests by approximately 300 ms.
- Restricts search to Israel and requests Hebrew results.
- Resolves the selected Place ID to latitude and longitude.
- Stores the five most recent successful searches in browser `localStorage` under `smd:recentAddressSearches`.

RouteLayer

- Receives GeoJSON `LineString` geometry returned by Mapbox.
- Draws optimized vehicle routes with vehicle-specific visual styling.
- Uses color, dash patterns, and route casing to improve visual distinction.

TaskTooltip

- Displays read-only operational information for a selected task.
- Omits optional fields when data is missing.
- May display installer, vehicle plate, customer details, address, time window, note, and optimized stop number.

LoginForm

- Collects user credentials.
- Submits authentication through the signIn Server Action.
- Uses React useState to manage errors, pending state, and submission.

6. Database Design

The main application table is public.service_tasks. It is a one-way operational mirror containing normalized service-task information supplied by the organization's upstream source system.

Field	Type	Purpose
id	string	Unique task identifier
address	string	Service address
lat	number	Latitude
lng	number	Longitude
time_window	string / null	Scheduled service window
vehicle_id	string	Assigned service vehicle
installer_name	string / null	Assigned installer
car_plate	string / null	Vehicle plate number
customer_name	string / null	Customer/contact name
customer_phone	string / null	Customer phone
status	TaskStatus	PENDING, COMPLETED, or FAILED
short_note	string / null	Operational note
scheduled_date	date string	Planned service date
created_at	timestamp	Record creation time

Supabase Authentication manages user identities separately from service_tasks. The current application does not require additional application tables for projects, comments, or organizations.

7. Database Indexing

The application frequently retrieves tasks by scheduled_date and orders the daily result by vehicle_id and time_window. A composite PostgreSQL index is implemented:

```
CREATE INDEX IF NOT EXISTS
service_tasks_scheduled_date_vehicle_id_time_window_idx
ON public.service_tasks (scheduled_date, vehicle_id, time_window);
```

The index supports the main daily-dashboard query and also helps date-range queries such as the four-day recommendation search. This reduces unnecessary scanning as the mirrored operational dataset grows.

8. CRUD Operations

The service-task data model is intentionally read-only from the application perspective.

Operation	Supported?	Reason
CREATE	No	The upstream operational system remains the source of truth.

READ	Yes	Required for dashboard and recommendations.
UPDATE	No	Changes in the dashboard mirror would not automatically propagate back to the upstream system.
DELETE	No	Operational records remain controlled by the authoritative source system.

This is an architectural rule rather than only a UI restriction. Database write privileges are revoked for normal application roles.

9. Server Actions

getDailyTasks(date)

1. Verify the authenticated Supabase session.
2. Retrieve only tasks whose scheduled_date equals the requested date.
3. Order tasks by vehicle_id and time_window.
4. Return task data together with an application-safe error state.

```
SELECT * FROM service_tasks
WHERE scheduled_date = selected_date
ORDER BY vehicle_id, time_window;
```

getRecommendations(lat, lng)

1. Validate coordinates received from the browser.
2. Verify authentication.
3. Calculate tomorrow according to the service timezone.
4. Retrieve tasks in the following four-day window.
5. Remove records with invalid coordinates.
6. Calculate Haversine distance to each candidate.
7. Remove tasks farther than 20 km.
8. Keep only the closest task for each vehicle_id + scheduled_date combination.
9. Sort recommendations by ascending distance.

The result is decision support only and does not modify any service-task record.

signIn()

- Authenticates using Supabase email/password authentication.
- Rejects missing credentials.
- Returns a generic authentication error.
- Validates the requested post-login redirect.
- Redirects after successful authentication.

signOut()

Terminates the Supabase authentication session and redirects the user to /login.

10. API Design

There are currently no custom Next.js /api Route Handlers. Internal backend operations use Server Actions.

Internal Backend Interface

```
getDailyTasks()
getRecommendations()
signIn()
signOut()
```

Supabase

- Authentication.
- Service-task retrieval.

Google Places API

Called from AddressSearch in the browser for autocomplete and Place Details coordinate resolution.

Mapbox Optimization API

Called from the browser to calculate a driving route. The response provides GeoJSON geometry, optimized waypoint order, driving distance, and estimated duration.

11. Route Optimization Logic

Daily tasks are grouped by vehicle_id. Only tasks with valid geographical coordinates can participate in route calculation.

A vehicle with fewer than two valid stops does not require an Optimization API request. Vehicles with multiple valid stops follow this flow:

```
Service tasks
|
Group by vehicle
|
Validate coordinates
|
Build Mapbox request
|
Optimization API
|
Route geometry + stop order
|
Render route + numbered markers
```

The current Mapbox Optimization API usage is limited to 12 coordinates per request. The first 12 routable stops participate in route optimization; additional valid stops remain visible as markers. Invalid-coordinate tasks are excluded from geographical calculations.

12. Route Caching and Request Management

Route fetching is isolated in src/hooks/useOptimizedRoutes.ts.

Each route request is identified using the vehicle ID together with its route coordinates. An in-memory cache stores resolved route results for the current browser page lifetime.

- Unrelated UI actions do not recalculate unchanged routes.
- Different vehicle routes are processed independently.
- AbortController cancels requests that are no longer relevant.
- Failed route calculations are cached temporarily to avoid continuous retries during React re-renders.

13. State Management

The application does not require an external global state-management library. State is divided according to responsibility.

URL State

The selected service day is stored in ?date=YYYY-MM-DD.

Server State

Daily task data is retrieved through Server Actions and passed from the Server Component to MapDashboard.

Client UI State

- Hidden vehicles.
- Focused vehicle.
- Hovered task.
- Pinned task.
- Map loading state.
- Searched draft location.
- Recommendation results.
- Recommendation loading state.

Vehicle and task selection state is scoped to the selected workday so UI choices from one day do not automatically affect another.

Route State

useOptimizedRoutes manages loading, ready/error state, GeoJSON geometry, stop numbers, distance, duration, cache, and in-flight requests.

Local Persistent State

Recent address searches are stored in browser localStorage and are not written to the organizational database.

14. Authentication and Authorization Architecture

Authentication is implemented using Supabase Auth. The application uses src/proxy.ts as a request-level authentication layer.

1. Refresh Supabase authentication cookies when required.
2. Verify authenticated claims.
3. Redirect unauthenticated visitors to /login.
4. Preserve the originally requested path and query string.
5. Redirect already-authenticated users away from /login.

Authentication is also checked inside data Server Actions, providing defense in depth.

```

Browser request
|
proxy.ts auth check
|
Protected page
|
Server Action
|
Auth checked again
|
Supabase

```

15. Database Authorization

Row Level Security is enabled on public.service_tasks. Anonymous users have SELECT revoked, while authenticated users receive read access through an explicit RLS policy.

```

anonymous      -> cannot read service_tasks
authenticated -> can read service_tasks

```


For mutations, PostgreSQL explicitly revokes INSERT, UPDATE, DELETE, and TRUNCATE from both anon and authenticated roles. This enforces the read-only operational-mirror architecture at the database layer.

16. Input Validation

Date Validation

isIsoDate() verifies that URL date parameters match YYYY-MM-DD and represent a real calendar date. Invalid values fall back to the current service date.

Geographic Coordinate Validation

Coordinates are checked for finite values and valid ranges before route or recommendation calculations: latitude [-90, 90], longitude [-180, 180].

Authentication Input

Login requires both email and password. Authentication failures use a generic error message to avoid account enumeration.

Redirect Validation

The post-login redirectTo value must be a same-origin relative path. External and protocol-relative destinations are rejected to prevent open redirects.

External API Response Validation

Mapbox responses are checked for successful status, a usable trip, a LineString geometry, and at least two route coordinates before rendering.

17. Error Handling

Daily Data Failure

- An empty task collection is returned.
- The page remains rendered.
- The error is shown to the user rather than crashing the route.

Mapbox Route Failure

- The failure affects only the corresponding vehicle.
- Service-task markers remain visible.
- Other vehicle routes continue to operate.

Google Places Failure

- The dashboard remains usable.
- Invalid suggestions are not displayed.
- Existing map and route functionality remains available.

Invalid Coordinates

Invalid task coordinates are excluded from geographical calculations while other valid tasks continue to function.

Missing Mapbox Configuration

If NEXT_PUBLIC_MAPBOX_TOKEN is missing, the application displays a configuration error instead of attempting to initialize an unusable map.

Browser Storage Failure

If localStorage is unavailable, address search still works; only Recent Searches persistence is lost.

18. User Experience Design

Hebrew and RTL

The main application layout is Hebrew-first and uses right-to-left direction. Date formatting and map labels are localized for Hebrew use.

Clear Vehicle Identification

Vehicle routes use multiple visual signals, including color, dash pattern, numbered stop markers, and labeled vehicle rows. Route identity therefore does not depend on color alone.

Immediate Task Details

Selecting a task displays operational details directly on the map without navigating away from the dashboard.

Vehicle Focus and Visibility

Dispatchers can focus on a vehicle or hide selected vehicles to simplify a crowded map.

Loading and Empty States

The interface provides feedback while loading data, calculating routes, or generating recommendations. A workday with no tasks displays an explicit empty state.

Recommendation Comparison

Selecting a recommendation changes the displayed workday while preserving the searched location, allowing comparison against the existing schedule before making a decision.

19. Environment Variables

The application requires runtime configuration through environment variables. Values are stored locally or in deployment configuration and are not committed to Git.

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
NEXT_PUBLIC_MAPBOX_TOKEN
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY
```

The End-to-End test environment additionally uses dispatcher credentials and a search address configured through test environment variables.

```
E2E_DISPATCHER_EMAIL
E2E_DISPATCHER_PASSWORD
E2E_SEARCH_ADDRESS
```

For deterministic E2E execution, `supabase/seed-e2e.sql` is run manually in the Supabase SQL Editor before the Playwright suite. It refreshes a dedicated recommendation fixture for the following day and does not change the permissions available to normal application users.

20. Key Technical Design Decisions

Upstream Operational System Remains the Source of Truth

The dashboard does not create a competing operational system. It reads normalized mirrored data from the organization's existing source system - for example, an ERP platform such as SAP - and supports dispatcher decision-making without becoming the system of record.

Read-Only Enforcement at Multiple Levels

Write actions are absent from the UI and Server Actions, and database write privileges are explicitly revoked.

Server-Side Operational Data Access

Supabase service-task retrieval is performed through authenticated Server Actions.

Client-Side Route Optimization

Mapbox route calculations run in the browser using a public client token, supporting interactive map behavior without unnecessary server requests.

Date Stored in the URL

The viewed workday is explicit, shareable, and compatible with browser navigation.

No Unnecessary Global State Library

React state, Server Component data, and URL state are sufficient for the current application complexity.

Graceful Degradation

Failure of an external routing service does not remove access to underlying operational task information.

21. Current Technical Limitations

- Mapbox Optimization is limited to 12 routable coordinates per vehicle request.
- Recommendation geographical filtering is performed in application logic after retrieving the four-day dataset.
- Recent searches are browser-local rather than synchronized between devices.
- Route results are cached only in the current browser session rather than shared between dispatchers.
- `service_tasks` is read-only and cannot commit scheduling changes back to the upstream ERP or operational source system.
- The recommendation algorithm is based primarily on geographical proximity and does not perform full workforce-capacity or time-window optimization.
- The source-system ingestion/synchronization adapter is not implemented in this project; the application assumes normalized service-task data is already available in Supabase.

These limitations are explicit design boundaries for the current project and provide clear directions for future development.